HTTP Requests and Responses

Learning Objective: Explain GET and POST methods and distinguish between 2xx, 4xx, and 5xx response status categories.

Target Level: Beginner	**Difficulty:** Intermediate	**Est. Duration:** 45 mins (Max: 45m)	**Status:** IN_REVIEW

1. Introduction (3 mins)

HTTP (Hypertext Transfer Protocol) is the foundation of data communication on the web. Every time you load a webpage, submit a form, or use a mobile app, your client sends an HTTP request and receives an HTTP response. For developers, mastering the anatomy of these messages is the first step in debugging APIs effectively. In this lesson, you will learn how GET and POST requests work, how to interpret 2xx, 4xx, and 5xx status codes, and how to apply this knowledge to diagnose real API issues. [RFC 9110: HTTP Semantics] defines the core semantics of these methods and status codes, while [MDN Web Docs] provides practical reference material.

[Applied QC Fix - Factual]:
Add a clarifying note that 422 and 429 are extension status codes (RFC 4918 and RFC 6585 respectively), or remove them from the RFC 9110-cited list and cite the appropriate extension RFCs.

2. Prerequisites (3 mins)

Before starting, you should be comfortable with:
- Navigating websites and understanding URLs (e.g., https://api.example.com/users)
- Opening browser Developer Tools and using the Network tab
- Basic JSON syntax (key-value pairs)
- Running simple commands in a terminal or using an API client like Postman or curl
No prior networking or backend experience is required.

3. Key Concepts (6 mins)

- **HTTP Request**: A message sent by a client to a server, containing a method, URL, headers, and optional body.
- **HTTP Response**: A message sent by a server back to the client, containing a status code, headers, and optional body.
- **GET**: Retrieves data from a server. It is safe and idempotent, meaning it should not modify server state and can be repeated without side effects.
- **POST**: Submits data to a server to create or update a resource. It typically includes a request body and is not idempotent.
- **Status Code Classes**: Three-digit codes grouped by first digit:
- **2xx**: Success (the request was received and processed)
- **4xx**: Client Error (the request contains bad syntax or cannot be fulfilled)
- **5xx**: Server Error (the server failed to fulfill a valid request)

[RFC 9110: HTTP Semantics] and [MDN Web Docs]

4. Detailed Explanations (10 mins)

GET Requests
GET requests retrieve a representation of a resource. Parameters are often sent in the URL query string (e.g., /search?q=api). Because GET is idempotent, browsers may cache responses, and clients can safely retry failed GET requests.

POST Requests
POST requests send data to the server, usually in the request body. Common Content-Type values include application/json and application/x-www-form-urlencoded. POST is used to create resources (e.g., submitting a signup form). Because POST can change server state, it is not idempotent—repeating the same POST may create duplicate resources.

Status Codes for Debugging
2xx Success
- 200 OK: The request succeeded. For GET, the resource is in the body. For POST, it often confirms processing.
- 201 Created: A new resource was successfully created (common after POST).
- 204 No Content: The request succeeded but there is no body to return.

4xx Client Error
- 400 Bad Request: The server could not understand the request due to invalid syntax.
- 401 Unauthorized: Authentication is required or has failed.
- 403 Forbidden: The client is authenticated but lacks permission.
- 404 Not Found: The requested resource does not exist.
- 422 Unprocessable Entity: The request was well-formed but semantically invalid (e.g., validation error).
- 429 Too Many Requests: Rate limiting has been applied.

5xx Server Error
- 500 Internal Server Error: An unexpected condition occurred on the server.
- 502 Bad Gateway: The server received an invalid response from an upstream server.
- 503 Service Unavailable: The server is temporarily overloaded or down for maintenance.
- 504 Gateway Timeout: The server did not receive a timely response from an upstream server.

Debugging Mindset
When debugging APIs, start by checking the status code class. A 4xx means inspect your request (URL, headers, body, authentication). A 5xx means the issue is likely on the server side and may require contacting the backend team or checking server logs. [RFC 9110: HTTP Semantics], [MDN Web Docs], [HTTP: The Definitive Guide]

5. Practical Examples (7 mins)

Example 1: GET Request with curl
```bash
curl -i https://jsonplaceholder.typicode.com/posts/1
```

Expected response: `HTTP/1.1 200 OK` with JSON body.

Example 2: POST Request with JSON
```bash
curl -i -X POST https://jsonplaceholder.typicode.com/posts -H 'Content-Type: application/json' -d
'{"title":"Hello","body":"API debugging","userId":1}'
```
Expected response: `HTTP/1.1 201 Created`.

Example 3: Interpreting Errors
- `404 Not Found` on GET /users/99999: The user ID does not exist. Check your request URL.
- `401 Unauthorized` on POST /login: Missing or invalid credentials. Check Authorization header.
- `500 Internal Server Error` on any request: Server-side bug. Retry later or check status page.

JavaScript Fetch Example
```javascript
fetch('https://api.example.com/data', {
method: 'POST',
headers: { 'Content-Type': 'application/json' },
body: JSON.stringify({ name: 'Ada' })
})
.then(response => {
if (!response.ok) throw new Error(`HTTP ${response.status}`);
return response.json();
});
```
[MDN Web Docs]

6. Real World Applications (5 mins)

In production systems, HTTP status codes drive monitoring, alerting, and user experience. Frontend applications inspect status codes to decide whether to show a success message, redirect to login, or display an error alert. Backend teams log 5xx errors to detect outages and use 4xx patterns to identify client-side bugs or malicious traffic. Tools like Postman, Insomnia, and browser DevTools let developers inspect live request/response cycles during API debugging. [Traversy Media], [HTTP: The Definitive Guide]

7. Hands-on Activities (6 mins)

Activity 1: Browser DevTools Inspection (3 minutes)
1. Open https://jsonplaceholder.typicode.com/posts/1 in your browser.
2. Open Developer Tools → Network tab.
3. Refresh the page and click the GET request.
4. Observe the Request URL, Method, Status Code (200), Response Headers, and Response Body.

Activity 2: curl Status Code Exploration (3 minutes)
Run the following commands and record the status codes:
- `curl -I https://jsonplaceholder.typicode.com/posts/1` (expect 200)
- `curl -I https://jsonplaceholder.typicode.com/posts/99999` (expect 404)
- `curl -X POST https://jsonplaceholder.typicode.com/posts -H 'Content-Type: application/json' -d '{}'` (expect 201)

Activity 3: Debugging Checklist
When you encounter an API failure:
1. Check the status code class (2xx, 4xx, 5xx).
2. If 4xx: verify URL, method, headers, and request body.
3. If 5xx: verify the service status page and retry after a delay.
4. Record the request and response for escalation if needed.
[MDN Web Docs]

8. Knowledge Check (3 mins)

Question 1
What is the primary difference between GET and POST?
Answer: GET retrieves data and is idempotent (no side effects), while POST submits data to create or modify resources and typically includes a request body.

Question 2
You receive a 404 Not Found response. Is this a client or server error?
Answer: Client error (4xx). The requested resource could not be found on the server, usually due to an incorrect URL or missing resource.

Question 3
Which status code indicates that a new resource was successfully created after a POST request?
Answer: 201 Created.

Question 4
You receive a 500 Internal Server Error while debugging an API. Where is the problem most likely located?
Answer: On the server side. The client request was valid, but the server encountered an unexpected condition.

Question 5
Which status code range should you investigate first if your API request returns invalid data despite a 200 OK?
Answer: 2xx means the request succeeded technically, so the issue is likely in the application logic or response data parsing, not the HTTP transport layer.

9. Summary (2 mins)

You now understand the core mechanics of HTTP requests and responses. GET is used to retrieve data safely, while POST is used to submit data and create resources. Status codes are grouped into classes: 2xx signals success, 4xx indicates client-side request problems, and 5xx signals server-side failures. When debugging APIs, always start by identifying the status code class to determine whether the issue lies in your request or the server. [RFC 9110: HTTP Semantics], [MDN Web Docs]

10. Additional Resources & References

- **YouTube Tutorial**: [HTTP Crash Course & Status Codes Explained](https://www.youtube.com/watch?v=iYM2zFP3Zn0) by Traversy Media — visual walkthrough of HTTP methods and status codes.
- **Book**: [HTTP: The Definitive Guide](https://books.google.com/books?id=f_a540C52z4C) by David Gourley and Brian Totty (O'Reilly Media) — comprehensive textbook reference on HTTP architecture, headers, and security.
- **Documentation**: [MDN Web Docs: HTTP](https://developer.mozilla.org/en-US/docs/Web/HTTP) — authoritative reference for request methods and response status codes.
- **Specification**: [RFC 9110: HTTP Semantics](https://www.rfc-editor.org/rfc/rfc9110.html) — formal IETF specification defining HTTP methods and status code classes.